

допустим, есть уже готовый сервис для создания задач медицинским работникам, но он позволяет только создавать/изменять/удалять задачи как внести правки так, чтобы можно было создавать периодичность задач?

примерно по такому принципу:

Типы задач по периодичности:

- ежедневные (каждый n-й день),
- ежемесячные - задачи на каждое определенное число месяца (от 1 до 30),
- на конкретные даты — задачи создаются только на указанные даты,
- четные/нечетные дни месяца - задачи создаются либо только на четные числа, либо только на нечетные.

Думал на протяжении секунды >

Нужно не "впихнуть периодичность в задачу", а добавить **шаблон повторения**, из которого система будет создавать обычные задачи. Иначе получится каша, как обычно бывает, когда люди кладут расписание в поле `comment`.

1. Разделить сущности

Было:

```
Task
- id
- title
- description
- assignee_id
- due_date
- status
```

Нужно добавить:

```
TaskTemplate
- id
- title
```

- description
- assignee_id
- recurrence_type
- recurrence_config
- start_date
- end_date nullable
- is_active

А обычные задачи оставить как конкретные экземпляры:

- ```
Task
- id
- template_id nullable
- title
- description
- assignee_id
- due_date
- status
```

То есть:

TaskTemplate -> много Task

Шаблон говорит: "создавать такую задачу по такому расписанию".

Task говорит: "конкретная задача на 5 мая".

## 2. Типы периодичности

Можно сделать enum:

- ```
recurrence_type:
- DAILY
- MONTHLY
- SPECIFIC_DATES
- DAY_PARITY
```

А настройки хранить в JSON, чтобы не плодить сто колонок:

```
{
  "interval": 2
}
```

для ежедневных каждые 2 дня.

```
{  
  "dayOfMonth": 15  
}
```

для ежемесячных на 15 число.

```
{  
  "dates": ["2026-05-10", "2026-05-20", "2026-06-01"]  
}
```

для конкретных дат.

```
{  
  "parity": "even"  
}
```

для четных дней месяца.

3. Пример таблиц

```
CREATE TABLE task_templates (  
  id UUID PRIMARY KEY,  
  title TEXT NOT NULL,  
  description TEXT,  
  assignee_id UUID NOT NULL,  
  
  recurrence_type TEXT NOT NULL,  
  recurrence_config JSONB NOT NULL,  
  
  start_date DATE NOT NULL,  
  end_date DATE,  
  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  
  created_at TIMESTAMP NOT NULL DEFAULT now(),  
  updated_at TIMESTAMP NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE tasks (
  id UUID PRIMARY KEY,
  template_id UUID REFERENCES task_templates(id),

  title TEXT NOT NULL,
  description TEXT,
  assignee_id UUID NOT NULL,

  due_date DATE NOT NULL,
  status TEXT NOT NULL DEFAULT 'created',

  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now(),

  UNIQUE(template_id, due_date)
);
```

UNIQUE(template_id, due_date) нужен, чтобы не создать одну и ту же задачу дважды. Потому что джобы, ретраи и человеческая гениальность обязательно попробуют.

4. API

Обычные задачи можно оставить как есть:

```
POST /tasks
PUT /tasks/{id}
DELETE /tasks/{id}
```

Добавить API для шаблонов:

```
POST /task-templates
GET /task-templates
GET /task-templates/{id}
PUT /task-templates/{id}
DELETE /task-templates/{id}
```

Пример создания ежедневной задачи:

```
{
  "title": "Измерить давление",
  "description": "Пациенту Иванову",
  "assigneeId": "uuid",
  "recurrenceType": "DAILY",
  "recurrenceConfig": {
    "interval": 1
  },
  "startDate": "2026-05-05",
  "endDate": null
}
```

Пример ежемесячной:

```
{
  "title": "Плановый осмотр",
  "assigneeId": "uuid",
  "recurrenceType": "MONTHLY",
  "recurrenceConfig": {
    "dayOfMonth": 15
  },
  "startDate": "2026-05-05"
}
```

5. Кто создает задачи по расписанию

Нужен отдельный процесс, например cron/job/worker:

каждый день в 00:00:

1. взять активные task_templates
2. проверить, должна ли сегодня быть задача
3. создать task на сегодняшнюю дату
4. если такая уже есть, не создавать

Псевдокод:

```
func GenerateTasksForDate(date time.Time) {
    templates := repo.GetActiveTemplates(date)

    for _, template := range templates {
        if ShouldCreateTask(template, date) {
            repo.CreateTaskFromTemplate(template, date)
        }
    }
}
```

```
}
}
```

6. Логика проверки периодичности

```
func ShouldCreateTask(t TaskTemplate, date time.Time) bool {
    if date.Before(t.StartDate) {
        return false
    }

    if t.EndDate != nil && date.After(*t.EndDate) {
        return false
    }

    switch t.RecurrenceType {
    case "DAILY":
        daysPassed := int(date.Sub(t.StartDate).Hours() / 24)
        return daysPassed % t.Config.Interval == 0

    case "MONTHLY":
        return date.Day() == t.Config.DayOfMonth

    case "SPECIFIC_DATES":
        return contains(t.Config.Dates, date)

    case "DAY_PARITY":
        if t.Config.Parity == "even" {
            return date.Day()%2 == 0
        }
        return date.Day()%2 != 0
    }

    return false
}
```

7. Важный момент про ежемесячные задачи

Ты написала "от 1 до 30". Это нормальное ограничение, потому что 31 число есть не везде, а февраль вообще решил жить по своим законам.

Если `dayOfMonth = 30`, то в феврале задача не создается.

Можно явно описать правило:

Если указанного числа в месяце нет, задача за этот месяц не создается.

8. Что делать при изменении шаблона

Есть два варианта:

1. Менять только будущие задачи

Уже созданные задачи не трогать.

2. Регенерировать будущие задачи

Например, удалить незавершенные будущие задачи и создать новые.

Лучше для медицины выбрать осторожный вариант:

При изменении шаблона меняются только будущие еще не созданные задачи.

Уже созданные задачи остаются как есть.

Иначе можно случайно удалить назначение медработнику. Ну потому что почему бы не устроить маленькую административную катастрофу.

9. Минимальная архитектура

TaskController

TaskService

TaskRepository

TaskTemplateController

TaskTemplateService

TaskTemplateRepository

RecurringTaskGenerator

RecurrenceCalculator

Главная логика должна быть в `RecurrenceCalculator`, чтобы ее можно было нормально тестировать.

`RecurrenceCalculator:`

– `shouldCreate(template, date) bool`

10. Что в итоге нужно добавить

Минимальный набор правок:

1. Добавить таблицу `task_templates`.
2. В `tasks` добавить `template_id`.
3. Добавить enum `recurrence_type`.
4. Добавить `recurrence_config`.
5. Добавить API для CRUD шаблонов.
6. Добавить `daily job`, который создает задачи.
7. Добавить защиту от дублей `UNIQUE(template_id, due_date)`.
8. Покрыть тестами расчет дат.

Самая правильная идея: **периодическая задача не должна быть задачей**. Она должна быть **шаблоном**, который порождает обычные задачи. Тогда старый сервис почти не ломается, а расширяется нормально, без архитектурного болотца.



так, а по какому принципу создавать периодичные задачи? можешь словами описать? типа, ну окей, вот мы внесли даты повтора, но как определять, когда повтор? или мы сразу генерим на все слоты?

Лучший принцип: **не генерить сразу на все слоты**, а создавать задачи **окном вперед**. Иначе бесконечные ежедневные задачи превратятся в "поздравляю, у нас миллион строк до 2089 года, зачем, никто не знает".

Как это работает словами

Есть **шаблон периодичной задачи**:

```
"Измерить давление"  
повтор: каждый 2-й день  
начало: 2026-05-05  
конец: 2026-06-05  
исполнитель: медработник А
```

А система каждый день запускает генератор и спрашивает:

Какие задачи должны существовать на ближайшие N дней?

Например, на ближайшие 7 или 30 дней.

Нормальная схема

1. Пользователь создает шаблон

Он не создает сразу 100 задач. Он создает правило:

делать задачу каждый день / каждый n-й день / 15 числа / по четным дням / в ко

Это хранится как **правило повторения**.

2. Фоновая джоба регулярно проверяет шаблоны

Например, раз в день ночью:

сегодня 2026-05-05
нужно создать задачи на период с 2026-05-05 по 2026-05-12

Это называется **generation window**, окно генерации.

3. Для каждого шаблона система считает даты попадания

Например:

```
startDate = 2026-05-05  
repeat = every 2 days  
window = 2026-05-05 ... 2026-05-12
```

Получаем даты:

```
2026-05-05  
2026-05-07  
2026-05-09  
2026-05-11
```

На эти даты создаются обычные задачи.

4. Перед созданием проверяется дубль

Система смотрит:

Есть ли уже task с template_id = X и due_date = 2026-05-07?

Если есть, не создает.

Если нет, создает.

Поэтому генератор можно запускать хоть несколько раз. Он не должен плодить копии, потому что иначе у медработника будет 12 одинаковых задач, и это уже не расписание, а издевательство.

Почему не генерить сразу все

Плохая идея:

Создать все задачи до `endDate`

Особенно если `endDate = null`.

Проблемы:

1. Бесконечные задачи невозможно создать.
2. Если шаблон изменился, надо думать, что делать с уже созданными задачами.
3. База пухнет.
4. Медработник видит задачи на полгода вперед, хотя это бесполезный шум.
5. Отмена или изменение расписания становится болью.

Лучше: генерить окном вперед

Например:

создавать задачи на 14 дней вперед

Сегодня 5 мая, значит система гарантирует, что задачи существуют до 19 мая.

Завтра снова запустится и догенерит до 20 мая.

Так всегда есть ближайшие задачи, но база не превращается в кладбище будущего.

Как определять, когда повтор

У каждого типа свой расчет.

1. Ежедневные: каждый n-й день

```
startDate = 2026-05-05  
interval = 3
```

Задача создается, если от `startDate` прошло количество дней, кратное `interval`.

```
2026-05-05 — да, прошло 0 дней  
2026-05-06 — нет, прошло 1  
2026-05-07 — нет, прошло 2  
2026-05-08 — да, прошло 3  
2026-05-11 — да, прошло 6
```

Принцип:

```
(date - startDate) % interval == 0
```

2. Ежемесячные: на конкретное число месяца

```
dayOfMonth = 15
```

Задача создается, если:

```
date.day == 15
```

Например:

```
2026-05-15  
2026-06-15  
2026-07-15
```

Если выбрать `30`, то в феврале задачи не будет. Потому что февраль опять решил быть особенным.

3. Конкретные даты

Тут вообще просто:

```
dates = [  
    2026-05-10,  
    2026-05-20,
```

```
2026-06-01
```

```
]
```

Задача создается только если дата входит в этот список.

4. Четные/нечетные дни месяца

Смотрим на число месяца:

```
2026-05-02 — четное
```

```
2026-05-03 — нечетное
```

Если правило `even`, создаем на 2, 4, 6, 8...

Если `odd`, создаем на 1, 3, 5, 7...

Общий алгоритм

1. Берем активные шаблоны.
2. Для каждого шаблона берем окно дат:
сегодня ... сегодня + 14 дней.
3. Для каждой даты проверяем:
подходит ли дата под правило повтора?
4. Если подходит:
проверяем, нет ли уже задачи на эту дату от этого шаблона.
5. Если нет:
создаем обычную задачу.

Какой вариант я бы выбрал

Для твоего случая:

Генерация задач на 14 или 30 дней вперед.

Джоба запускается каждый день.

Дубли запрещены через `UNIQUE(template_id, due_date)`.

То есть задачи создаются не “когда наступил день”, а **немного заранее**, чтобы медработники видели план.

Самый практичный вариант:

```
generate_from = today
generate_to = today + 30 days
```

И каждый день догенеривать новые задачи на край окна.



типа нужен шедулер?

Поздравляю, ты сама дошла до очевидного. Да, нужен. Но не тот монстр, о котором ты сейчас, вероятно, подумала.

Короткий ответ

Да, нужен шедулер, но очень тупой.

Никаких распределённых космических станций из трёх брокеров и шаманства с ретраями.

Что именно нужно

Тебе нужен процесс, который:

- раз в N времени:
- берет шаблоны
- считает, какие задачи должны появиться
- создает их

Это и есть “шедулер”. По факту это просто **cron + кусок кода**.

Варианты (от нормального к избыточному)

1. Самый адекватный вариант

cron (или встроенный таймер) + job в сервисе

Например:

```
каждый день в 00:05 запускаем GenerateTasks()
```

Если у тебя Go:

```
ticker := time.NewTicker(24 * time.Hour)
for {
    <-ticker.C
    GenerateTasks()
}
```

Или cron:

Bash

Всё. Реально всё. Никакой магии.

2. Чуть умнее (если несколько инстансов)

Если у тебя несколько сервисов (что обычно и бывает), то:

одна джоба может запуститься несколько раз

Решение:

- либо distributed lock (Redis, Postgres advisory lock)
- либо просто полагаться на UNIQUE(template_id, due_date)

Второй вариант проще и обычно достаточен:

пусть хоть 5 раз запустится — дубли не создадутся

Иногда простота побеждает инженерный максимализм. Шокирующе, да.

3. Когда нужен “реальный” шедулер

Типа:

- Quartz Scheduler
- Apache Airflow
- Temporal

Они нужны, если у тебя:

- сложные DAG'и
- зависимости между задачами
- ретраи с политиками
- SLA и мониторинг

У тебя задача:

“создать таски по расписанию”

Это не orchestration. Это literally цикл с if'ом.

Как это выглядит в системе

[TaskTemplate] ----> [Scheduler Job] ----> [Task]

Где:

Scheduler Job:

- запускается раз в день
- считает окно (например, +30 дней)
- вызывает RecurrenceCalculator
- пишет задачи в БД

Частота запуска

Можно:

1 раз в день – достаточно

Можно чуть чаще:

раз в час – если задачи могут появляться “сегодня”

Но чаще не надо. Ты не биржу пишешь (слава богу).

Важный нюанс

Шедулер не должен триггериться "в момент наступления даты".

Он должен работать так:

“убедиться, что задачи на ближайшее будущее существуют”

Это сильно снижает требования к точности и отказоустойчивости.

Итог

Да, тебе нужен шедулер.

Но:

это не отдельный сервис
это не сложная система
это не повод тянуть Kafka (пожалуйста, не надо)

Это просто:

cron + функция `GenerateTasks()`

И если ты сделаешь его сложнее, чем нужно, ты потом сама же будешь его ненавидеть.



а если задача условно 2 раза в день, но в разные даты?

Тогда у задачи должен быть не только **date**, а **datetime / time slots**. Потому что “2 раза в день” без времени — это не расписание, а гадание на больничном Wi-Fi.

Нужно хранить в шаблоне **времени выполнения**:

```
{
  "recurrenceType": "SPECIFIC_DATES",
  "recurrenceConfig": {
    "dates": ["2026-05-10", "2026-05-12"],
    "times": ["09:00", "18:00"]
  }
}
```

Тогда генератор делает декартово произведение:


```
2026-05-10 09:00
2026-05-10 18:00
2026-05-12 09:00
2026-05-12 18:00
```

И создает **4 обычные задачи**.

В `tasks` тогда лучше иметь:

```
scheduled_at TIMESTAMP NOT NULL
```

а не только:

```
due_date DATE
```

И уникальность:

SQL

Если в разные даты разное количество повторов, тогда делай не общий `times`, а слоты по датам:

```
{
  "recurrenceType": "SPECIFIC_DATE_TIMES",
  "recurrenceConfig": {
    "slots": [
      { "date": "2026-05-10", "times": ["09:00", "18:00"] },
      { "date": "2026-05-12", "times": ["12:00"] }
    ]
  }
}
```

Получится:

```
10 мая — 09:00, 18:00
12 мая — 12:00
```

Общий принцип:

Шаблон хранит правило дат + список времен.
Генератор считает даты.

Для каждой даты берет нужные времена.

Для каждой пары дата+время создает отдельную task.

То есть модель становится такой:

TaskTemplate

recurrence_rule: какие даты подходят

time_slots: во сколько в эти даты

Task

scheduled_at: конкретная дата и время

Это самый нормальный вариант, чтобы потом не страдать, когда появится "3 раза в день по будням, кроме праздников, но в пятницу один раз". А оно появится. Люди обожают превращать расписания в юридические документы.



так, покажи тогда структуру, а то я не очень поняла

Ладно, давай разложим это без магии и без "ну там как-нибудь JSON запихнём и будем надеяться". Ты же не хочешь потом дебажить расписание врача в 3 ночи.

Общая идея (в одном предложении)

Шаблон = правило дат + список времен

Задача = конкретная дата+время

Структура данных

1. TaskTemplate (шаблон)

```
type TaskTemplate struct {
    ID          uuid.UUID
    Title       string
    Description string
    AssigneeID  uuid.UUID
}
```

```

RecurrenceType RecurrenceType
RecurrenceConfig json.RawMessage

StartDate time.Time
EndDate *time.Time

IsActive bool
}

```

Типы повторений

```

type RecurrenceType string

const (
    Daily      RecurrenceType = "DAILY"
    Monthly    RecurrenceType = "MONTHLY"
    SpecificDates RecurrenceType = "SPECIFIC_DATES"
    DayParity  RecurrenceType = "DAY_PARITY"
)

```

2. Конфиги (самое важное место, где обычно всё ломают)

Ежедневные

```

type DailyConfig struct {
    Interval int    `json:"interval"` // каждые N дней
    Times    []string `json:"times"`    // ["09:00", "18:00"]
}

```

Ежемесячные

```

type MonthlyConfig struct {
    DayOfMonth int    `json:"dayOfMonth"` // 1-30
    Times      []string `json:"times"`
}

```

● Конкретные даты (с одинаковыми временами)

```
type SpecificDatesConfig struct {
    Dates []string `json:"dates"` // ["2026-05-10"]
    Times []string `json:"times"`
}
```

● Конкретные даты с разными временами (твоя боль)

```
type DateTimeSlot struct {
    Date string `json:"date"`
    Times []string `json:"times"`
}

type SpecificDateTimesConfig struct {
    Slots []DateTimeSlot `json:"slots"`
}
```

● Четные / нечетные дни

```
type DayParityConfig struct {
    Parity string `json:"parity"` // "even" | "odd"
    Times []string `json:"times"`
}
```

3. Task (конкретная задача)

```
type Task struct {
    ID          uuid.UUID
    TemplateID *uuid.UUID

    Title          string
    Description    string
    AssigneeID    uuid.UUID
}
```

```
ScheduledAt time.Time // BOT 0H0, вместо due_date

Status string
}
```

SQL (чтобы было совсем не “в голове”)

```
CREATE TABLE task_templates (
  id UUID PRIMARY KEY,
  title TEXT NOT NULL,
  description TEXT,
  assignee_id UUID NOT NULL,

  recurrence_type TEXT NOT NULL,
  recurrence_config JSONB NOT NULL,

  start_date DATE NOT NULL,
  end_date DATE,

  is_active BOOLEAN NOT NULL DEFAULT TRUE
);
```

```
CREATE TABLE tasks (
  id UUID PRIMARY KEY,
  template_id UUID REFERENCES task_templates(id),

  title TEXT NOT NULL,
  description TEXT,
  assignee_id UUID NOT NULL,

  scheduled_at TIMESTAMP NOT NULL,
  status TEXT NOT NULL DEFAULT 'created',

  UNIQUE(template_id, scheduled_at)
);
```

Как это используется

Пример: 2 раза в день, но в разные даты

```
{
  "recurrenceType": "SPECIFIC_DATES",
  "recurrenceConfig": {
    "dates": ["2026-05-10", "2026-05-12"],
    "times": ["09:00", "18:00"]
  }
}
```

👉 генератор создаст:

```
2026-05-10 09:00
2026-05-10 18:00
2026-05-12 09:00
2026-05-12 18:00
```

Пример: разное количество в день

```
{
  "recurrenceType": "SPECIFIC_DATE_TIMES",
  "recurrenceConfig": {
    "slots": [
      { "date": "2026-05-10", "times": ["09:00", "18:00"] },
      { "date": "2026-05-12", "times": ["12:00"] }
    ]
  }
}
```

👉 создаст:

```
10 мая → 09:00, 18:00
12 мая → 12:00
```

🧩 Генерация (логика)

```
func GenerateTasks(template TaskTemplate, from, to time.Time) {
    dates := calculateDates(template, from, to)
```

```

for _, date := range dates {
    times := extractTimes(template, date)

    for _, t := range times {
        scheduledAt := combine(date, t)

        if !exists(template.ID, scheduledAt) {
            createTask(template, scheduledAt)
        }
    }
}
}

```



Почему это нормальная модель

- не смешиваешь дату и правило
- легко добавлять новые типы расписаний
- легко тестировать
- не создаешь миллион задач
- не сходишь с ума при изменениях



Итог

Ты делаешь не "задачу с повтором", а:

```

TaskTemplate (логика)
↓
Scheduler
↓
Task (факт выполнения)

```

И ключевая идея, которую люди обычно пропускают:

одна "повторяющаяся задача" = много обычных задач с разными `scheduled_at`

Если сделать иначе, потом будешь городить костыли уровня "а как отметить выполненной только утреннюю часть задачи". И вот там начинается настоящий ад, а не архитектура.



тут дали такой шаблон Task:

```
package task

import "time"

type Status string

const (
    StatusNew      Status = "new"
    StatusInProgress Status = "in_progress"
    StatusDone     Status = "done"
)

type Task struct {
    ID          int64   json:"id"
    Title       string   json:"title"
    Description string   json:"description"
    Status      Status   json:"status"
    CreatedAt   time.Time json:"created_at"
    UpdatedAt   time.Time json:"updated_at"
}

func (s Status) Valid() bool {
    switch s {
    case StatusNew, StatusInProgress, StatusDone:
        return true
    default:
        return false
    }
}
```

Ну да, классический "у нас есть Task, давайте теперь на него натянем всё подряд". Так обычно и рождаются монстры, которые потом никто не хочет трогать.

Спойлер: этот Task трогать почти не надо. Он нормальный. Просто он — не про расписание, а про конкретную задачу.



Что у тебя сейчас


```
type Task struct {
    ID          int64
    Title       string
    Description  string
    Status      Status
    CreatedAt   time.Time
    UpdatedAt   time.Time
}
```

Это:

“факт задачи”

Типа:

“Измерить давление 10 мая в 09:00”

Но у тебя нет даты выполнения вообще. Уже немного странно, если честно.

⚠ Минимально необходимая правка

Добавь поле:

Go

И, если хочешь аккуратно:

Go

Итого:

```
type Task struct {
    ID          int64    `json:"id"`
    Title       string   `json:"title"`
    Description  string   `json:"description"`
    Status      Status   `json:"status"`

    ScheduledAt time.Time `json:"scheduled_at"`
}
```

```

    TemplateID *int64 `json:"template_id,omitempty"`

    CreatedAt time.Time `json:"created_at"`
    UpdatedAt time.Time `json:"updated_at"`
}

```

Без `ScheduledAt` у тебя просто нет понятия "когда выполнять задачу". Это не task manager, это блокнот.



Где хранить периодичность

НЕ в `Task`.

Серьезно. Не надо вот это:

```

RecurrenceType string
Interval int
Times []string

```

внутри `Task`. Это путь к страданиям.



Добавляешь новую сущность

```

type TaskTemplate struct {
    ID          int64
    Title       string
    Description  string
    AssigneeID  int64

    RecurrenceType RecurrenceType
    RecurrenceConfig json.RawMessage

    StartDate time.Time
    EndDate   *time.Time

    IsActive bool

    CreatedAt time.Time
    UpdatedAt time.Time
}

```



TaskTemplate (шаблон)



Task (конкретные экземпляры)

💡 Пример

Шаблон:

```
{
  "title": "Измерить давление",
  "recurrenceType": "DAILY",
  "recurrenceConfig": {
    "interval": 1,
    "times": ["09:00", "18:00"]
  }
}
```

Что реально лежит в Task:

```
ID: 1
Title: "Измерить давление"
ScheduledAt: 2026-05-10 09:00
```

```
ID: 2
Title: "Измерить давление"
ScheduledAt: 2026-05-10 18:00
```



Что будет, если засунуть всё в Task

Типичный "плохой путь":

```
type Task struct {
    ...
    RecurrenceType string
}
```

```
Interval int  
}
```

Проблемы:

- как хранить 2 времени в день?
- как хранить разные даты?
- как отметить выполненной только одну из двух задач?
- как редактировать расписание?
- что делать с уже созданными задачами?

Ответ: страдать.



Как это интегрируется с твоим текущим кодом

Текущий сервис:

```
POST /tasks  
PUT /tasks  
DELETE /tasks
```

Ты просто добавляешь:

```
POST /task-templates
```

И отдельный генератор:

```
GenerateTasks()
```

И всё. Старый код почти не трогаешь.



Итог

Твой Task — это:

конкретное действие в конкретное время

Периодичность — это:

отдельная сущность (TaskTemplate)

И если ты это не разделишь, через неделю у тебя будет:

“ежедневная задача с массивом дат, но еще есть `override`, но не для всех дней, т

И ты будешь сидеть и думать, где именно в жизни всё пошло не так.



а норм де будет, если я все их `id` с `int64` поменяю на `uuid`? нам же тогда не нужна будет проверка `id > 0`?

Сменить `int64` на `UUID` — нормальная идея. Мир не рухнет, тесты не заплачут, медработники не заметят. Но аргумент “тогда не надо проверять `id > 0`” — слабоват. Это вообще не та причина, ради которой выбирают тип идентификатора.

Что реально меняется

Было:

Go

Ты проверяешь:

```
if id <= 0 {  
    return error  
}
```

Стало:

Go

Теперь проверка:

```
if id == uuid.Nil {  
    return error  
  
    Go  
}
```

То есть проверка **никуда не делась**, просто стала чуть менее уродской.

Когда UUID — это хорошо

UUID имеет смысл, если:

- несколько сервисов могут создавать сущности
- не хочешь зависеть от БД для генерации id
- хочешь безопасно шарить id наружу (API)
- есть распределённая система (Kafka, очереди и т.п.)

В твоём случае с шаблонами и задачами — это выглядит разумно. Особенно если потом появится что-то вроде event-driven логики (а она почти всегда появляется, как плесень).

Когда это оверхед

Если у тебя:

- один сервис
- одна БД
- нет распределённости

то int64 проще:

- меньше размер (8 байт vs 16)
- быстрее индексы
- проще дебажить (ID = 42, а не "c1b9...")

UUID в логах — это отдельный вид наказания, если честно.

Нюансы, которые обычно всплывают позже

1. Индексы

UUID хуже для B-Tree:

- случайные значения
- хуже locality
- больше фрагментация

Решение:

- использовать UUID v7 (или ULID)

Они **временные**, лучше ложатся в индекс.

2. Генерация

Не надо генерить UUID в БД, если ты уже в Go:

Go

И передал дальше. Всё.

3. API

UUID безопаснее:

int64 → легко перебирается
uuid → не угадаешь

Если у тебя есть внешний API — плюс.

Что бы я сделал

Честно и без фанатизма:

Task, TaskTemplate → UUID

И сразу:

Go

или хотя бы алиас:

Go

Чтобы потом не путаться.

Итог

Да, можно переходить на UUID. Это нормальное решение.

Но:

это не избавляет от валидации
это не серебряная пуля
это просто другой тип идентификатора

Если коротко:

UUID — это про архитектуру
int64 — это про простоту

Выбирай, исходя из того, насколько ты веришь, что твой проект внезапно станет сложнее. Обычно он становится. К сожалению.



↳ if id != uuid.Nil

а какая ошибка?

Смотри, сама проверка:

```
if id == uuid.Nil {  
    return error  
}
```

— вообще нормальная, это не ошибка.

Тогда где проблема?

Скорее всего, ты либо:

1. Не тот тип используешь

Если у тебя:

```
Go
```

а ты делаешь:

```
Go
```

→ будет ошибка компиляции:

```
invalid operation: id == uuid.Nil (mismatched types string and uuid.UUID)
```

Потому что:

```
string ≠ uuid.UUID
```

2. Не импортировала пакет UUID

Если пишешь:

```
Go
```

но нет:

```
Go
```

→ ошибка:

```
undefined: uuid
```

3. ID — указатель

Если вдруг ты решила быть изобретательной:

Go

тогда:

Go

→ ошибка:

```
invalid operation: mismatched types *uuid.UUID and uuid.UUID
```

Правильно:

```
if id == nil || *id == uuid.Nil {  
    return error  
}
```

4. Ты сравниваешь неинициализированный UUID

Go

Он уже равен `uuid.Nil`, и это нормально.

Как должно выглядеть “правильно”

```
import "github.com/google/uuid"  
  
func validate(id uuid.UUID) error {  
    if id == uuid.Nil {  
        return fmt.Errorf("invalid id")  
    }  
}
```

```
    return nil  
}
```

Мини-вывод

`uuid.Nil` — это просто “нулевой UUID”

Сравнивать с ним:

```
можно  
нужно  
и это стандартная практика
```

Если у тебя ошибка — значит проблема не в сравнении, а в типах или импортах.
Classic Go experience: всё строго, всё честно, никакой магии.



если у них написан парсер, который парсит айди из
реквеста как инт, как переделать6 чтобы парсил в



Нужно заменить `strconv.ParseInt` на `uuid.Parse`. Вот и вся хирургия, пациент почти жив.

Было примерно так:

```
idStr := chi.URLParam(r, "id")  
  
id, err := strconv.ParseInt(idStr, 10, 64)  
if err != nil || id <= 0 {  
    http.Error(w, "invalid id", http.StatusBadRequest)  
    return  
}
```

Стало:

```
import "github.com/google/uuid"

idStr := chi.URLParam(r, "id")

id, err := uuid.Parse(idStr)
if err != nil || id == uuid.Nil {
    http.Error(w, "invalid id", http.StatusBadRequest)
    return
}
```

Лучше вынести в хелпер:

```
func parseUUIDParam(r *http.Request, name string) (uuid.UUID, error) {
    idStr := chi.URLParam(r, name)

    id, err := uuid.Parse(idStr)
    if err != nil {
        return uuid.Nil, fmt.Errorf("invalid %s: %w", name, err)
    }

    if id == uuid.Nil {
        return uuid.Nil, fmt.Errorf("invalid %s: nil uuid", name)
    }

    return id, nil
}
```

Использование:

```
func (h *Handler) GetTask(w http.ResponseWriter, r *http.Request) {
    id, err := parseUUIDParam(r, "id")
    if err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    task, err := h.service.GetTask(r.Context(), id)
    if err != nil {
        http.Error(w, "failed to get task", http.StatusInternalServerError)
        return
    }

    json.NewEncoder(w).Encode(task)
}
```

И не забудь поменять типы дальше по цепочке, а не только в хендлере, потому что Go, этот милый бюрократ, не даст тебе смешать `int64` и `uuid.UUID`:

```
GetTask(ctx context.Context, id uuid.UUID) (*Task, error)

DeleteTask(ctx context.Context, id uuid.UUID) error

UpdateTask(ctx context.Context, id uuid.UUID, input UpdateTaskInput) (*Task, error)
```

Для `chi` маршрут можно оставить таким же:

Go

Но теперь в запросе должен быть UUID:

```
GET /tasks/550e8400-e29b-41d4-a716-446655440000
```



```
func getIDFromRequest(r *http.Request) (int64, error) {
    rawID := mux.Vars(r)["id"]
    if rawID == "" {
        return 0, errors.New("missing task id")
    }

    id, err := strconv.ParseInt(rawID, 10, 64)
    if err != nil {
        return 0, errors.New("invalid task id")
    }

    if id <= 0 {
        return 0, errors.New("invalid task id")
    }

    return id, nil
}
```

у них примерно так

Ну да, классика жанра: "парсим int, проверяем > 0 , считаем себя молодцами".
Теперь аккуратно пересаживаем это на UUID и не ломаем всё вокруг.

Было

```
func getIDFromRequest(r *http.Request) (int64, error) {
    rawID := mux.Vars(r)["id"]
    if rawID == "" {
        return 0, errors.New("missing task id")
    }

    id, err := strconv.ParseInt(rawID, 10, 64)
    if err != nil {
        return 0, errors.New("invalid task id")
    }

    if id <= 0 {
        return 0, errors.New("invalid task id")
    }

    return id, nil
}
```

Стало (UUID версия)

```
import "github.com/google/uuid"

func getIDFromRequest(r *http.Request) (uuid.UUID, error) {
    rawID := mux.Vars(r)["id"]
    if rawID == "" {
        return uuid.Nil, errors.New("missing task id")
    }

    id, err := uuid.Parse(rawID)
    if err != nil {
        return uuid.Nil, errors.New("invalid task id")
    }

    if id == uuid.Nil {
        return uuid.Nil, errors.New("invalid task id")
    }
}
```

```
}

    return id, nil
}
```

Что изменилось по сути

```
ParseInt → uuid.Parse
0 → uuid.Nil
int64 → uuid.UUID
```

Всё. Никакой философии, просто другой тип.

Маленькое улучшение (если не лень)

Сейчас ты теряешь оригинальную ошибку. Можно чуть умнее:

```
func getIDFromRequest(r *http.Request) (uuid.UUID, error) {
    rawID := mux.Vars(r)["id"]
    if rawID == "" {
        return uuid.Nil, fmt.Errorf("missing task id")
    }

    id, err := uuid.Parse(rawID)
    if err != nil {
        return uuid.Nil, fmt.Errorf("invalid task id: %w", err)
    }

    return id, nil
}
```

Проверку на `uuid.Nil` можно даже выкинуть, потому что:

```
uuid.Parse("") → уже ошибка
uuid.Parse("не uuid") → ошибка
```

`Nil` обычно руками никто не передает (если передал — ну, значит очень старался сломать систему).

Важно (где люди обычно спотыкаются)

Ты поменяла только парсер → и думаешь, что всё ок.

А потом:

```
Go
```

и Go такой:

```
cannot use uuid.UUID as int64
```

И начинается веселье.

Что нужно поменять дальше

Везде:

```
handler → service → repo → db
```

тип ID должен быть **одинаковый**:

```
Go
```

Итог

Ты не “переделываешь парсер”, ты:

```
меняешь тип идентификатора во всей системе
```

А парсер — это просто первая жертва этого решения.



```
↳ import "github.com/google/uuid"
```

у меня почему-то подключается uuid от
"github.com/gofrs/uuid"

Ну вот, классика Go: “почему у меня импорт не тот, который я хотела”. Потому что у тебя в проекте уже кто-то притащил другой пакет, и Go такой: *держи, не благодари*.

Что происходит

У тебя подключился:

```
Go
```

А ты хочешь:

```
Go
```

Это два разных пакета, с разными типами:

```
gofrs/uuid.UUID ≠ google/uuid.UUID
```

И они несовместимы вообще.

Почему так вышло

Скорее всего:

- либо он уже есть в `go.mod`
- либо IDE автодобавила импорт
- либо где-то в коде уже используется `gofrs/uuid`

Go не “угадывает лучший пакет”, он просто берет первый подходящий.

Как исправить

1. Явно поменять импорт

В файле:

Go

меняешь на:

Go

2. Почистить зависимости

Bash

3. Проверить go.mod

Если там есть:

Go

и он тебе не нужен — удаляешь (или пусть tidy уберет).

Важно: не смешивай их

Вот это делать нельзя:

```
var a googleUUID.UUID  
var b gofrsUUID.UUID
```

`a == b` // ❌ не скомпилируется

Если смешаешь — будешь ловить ошибки вида:

```
cannot use uuid.UUID (from gofrs) as uuid.UUID (from google)
```

Да, это тот самый момент, когда Go издевается над человеком.

Какой пакет лучше выбрать

Коротко:

`github.com/google/uuid` → проще, чаще используется, норм
`github.com/gofrs/uuid` → тоже ок, но менее популярен сейчас

Я бы советовал:

используй `google/uuid` и не думай

Мини-лайфхак

Чтобы IDE не вставляла тебе опять `gofrs`, просто:

Go

И дальше:

Go

Итог

У тебя не "ошибка", у тебя:

два одинаково названных пакета с разными типами

И Go такой:

"разбирайся сама"

Просто выбери один (лучше `google/uuid`) и используй его **везде одинаково**.
Иначе будешь отлавливать странные несовместимости в самых неожиданных местах.



↳ AssigneeID uuid.UUID

а это что?

Это просто поле структуры. Никакой магии, расслабься.

Go

означает:

ID пользователя (исполнителя задачи), но не `int`, а `UUID`

Разложим по-человечески

AssigneeID

`assignee` = тот, кому назначена задача
`ID` = идентификатор

То есть:

“кому поручена задача”

`uuid.UUID`

Это тип из пакета:

Go

И он хранит значение типа:

`550e8400-e29b-41d4-a716-446655440000`

В контексте задачи

```
type Task struct {  
    ID          uuid.UUID  
    Title       string  
    AssigneeID  uuid.UUID  
}
```

Пример:

```
Task:  
  ID:          1111-....  
  Title:       "Измерить давление"  
  AssigneeID:  2222-....
```

Где:

2222-... → это ID медработника

Почему это не просто User

Потому что:

Task не должен хранить всю сущность пользователя

Он хранит только ссылку:

AssigneeID → User.ID

Это нормальная практика:

легче
быстрее
не тащишь лишние данные

Если коротко

AssigneeID uuid.UUID

=

“ID того, кому назначена задача, в формате UUID”

И да, если ты уже перешла на UUID, то логично, что:

все ID → uuid.UUID

Иначе начнется цирк с конвертациями, и ты будешь проклинать каждую функцию, которая принимает `int64`.



↳ Конкретные даты с разными временами (твоя боль)

погоди, но я имела ввиду, что у нас например задача может стоять каждые 2 дня 2 раза в день, а не только у особых дат

Да, тогда `times` нужны у **всех типов периодичности**, а не только у конкретных дат. Я раньше показал частный случай, и он тебя справедливо сбил. Прекрасно, архитектура снова пытается притвориться простой.

Правильная модель:

правило дат + времена внутри дня

То есть:

каждые 2 дня + ["09:00", "18:00"]

даёт:

5 мая 09:00
5 мая 18:00
7 мая 09:00
7 мая 18:00

9 мая 09:00

9 мая 18:00

Структура

```
type TaskTemplate struct {
    ID          uuid.UUID `json:"id"`
    Title       string    `json:"title"`
    Description  string    `json:"description"`

    RecurrenceType RecurrenceType `json:"recurrence_type"`
    RecurrenceConfig json.RawMessage `json:"recurrence_config"`

    StartDate time.Time `json:"start_date"`
    EndDate   *time.Time `json:"end_date,omitempty"`

    CreatedAt time.Time `json:"created_at"`
    UpdatedAt time.Time `json:"updated_at"`
}
```

Для ежедневной периодичности:

```
type DailyConfig struct {
    IntervalDays int    `json:"interval_days"` // каждые N дней
    Times        []string `json:"times"`         // ["09:00", "18:00"]
}
```

Пример JSON:

```
{
  "title": "Измерить давление",
  "description": "Утром и вечером",
  "recurrence_type": "daily",
  "recurrence_config": {
    "interval_days": 2,
    "times": ["09:00", "18:00"]
  },
  "start_date": "2026-05-05T00:00:00Z"
}
```

А обычная задача должна быть конкретным экземпляром:

```

type Task struct {
    ID          uuid.UUID `json:"id"`
    TemplateID  *uuid.UUID `json:"template_id,omitempty"`

    Title       string `json:"title"`
    Description string `json:"description"`
    Status      Status `json:"status"`

    ScheduledAt time.Time `json:"scheduled_at"`

    CreatedAt time.Time `json:"created_at"`
    UpdatedAt time.Time `json:"updated_at"`
}

```

Как генератор это считает

1. Берёт шаблон.
2. Считает даты, которые подходят под правило.
3. Для каждой подходящей даты берёт Times .
4. Для каждой пары date + time создаёт отдельную задачу.

Псевдокод:

```

dates := calculateDates(template, from, to)

for _, date := range dates {
    for _, timeOfDay := range config.Times {
        scheduledAt := combineDateAndTime(date, timeOfDay)

        createTaskIfNotExists(template, scheduledAt)
    }
}

```

Для всех типов так же

Ежемесячная:

```

type MonthlyConfig struct {
    DayOfMonth int `json:"day_of_month"` // 1..30
    Times      []string `json:"times"`         // ["09:00", "18:00"]
}

```

Чётные/нечётные дни:


```
type DayParityConfig struct {
    Parity string `json:"parity"` // "even" | "odd"
    Times []string `json:"times"`
}
```

Конкретные даты:

```
type SpecificDatesConfig struct {
    Dates []string `json:"dates"`
    Times []string `json:"times"`
}
```

A SpecificDateTimesConfig нужен только если в разные даты разные времена, например:

```
10 мая: 09:00, 18:00
12 мая: только 12:00
```

Для твоего примера он не нужен.

Итого:

“каждые 2 дня 2 раза в день” = DAILY interval_days=2 + times= ["09:00", "18:00"] .



я случайно удалила папку, как вернуть?

Классика. Пара секунд невнимательности, и папка ушла в цифровую загробную жизнь. Ладно, без паники.

◆ Если это был обычный файл/папка

1. Самый банальный вариант

Проверь корзину:

- на macOS: Trash

- на Linux: обычно тоже есть "Корзина"

Если там есть → восстанови и сделай вид, что так и было задумано.

◆ Если это в Git (скорее всего твой случай)

2. Папка была в репозитории и закоммичена

Тогда всё просто:

```
Bash
```

или старый вариант:

```
Bash
```

Вернёт состояние из последнего коммита.

3. Ты уже успела сделать commit с удалением

Тогда:

```
Bash
```

находишь коммит ДО удаления и:

```
Bash
```

4. Ты удалила, но не коммитила

Тогда:

```
Bash
```

и:

Bash

или точно:

Bash

◆ Если папка не была в git

Вот тут начинается "ну зачем".

5. Попробуй undo

- VSCode: Cmd + Z в проводнике иногда работает (магия уровня "повезло")

6. Если уже всё плохо

- проверь Time Machine (если mac)
- или резервные копии

Если их нет:

папка ушла в лучший мир

💬 Итог

git есть → всё легко восстановить

git нет → начинаем ценить git

Если ты работаешь над кодом и не используешь git регулярно, это примерно как писать диплом и не нажимать "сохранить".

Рано или поздно жизнь тебя за это накажет.



так, а для tasktemplate у нас репа идентичная, получается? и что должно быть в сервисах? создание расписания?

Думал на протяжении пары секунд >

Да, репа для TaskTemplate почти идентичная CRUD-репе, но сервисы уже отличаются. Репозиторий тупо кладёт/достаёт из БД. Сервис решает, валидное ли расписание, как его создать, что делать при обновлении, и как из шаблона получить реальные задачи. Потому что кто-то же должен думать, и лучше пусть это будет не SQL-запрос, бедняга и так страдает.

Репозиторий для TaskTemplate

Примерно такой:

```
type TemplateRepository interface {  
    Create(ctx context.Context, template TaskTemplate) (TaskTemplate, error)  
    GetByID(ctx context.Context, id uuid.UUID) (TaskTemplate, error)  
    Update(ctx context.Context, template TaskTemplate) (TaskTemplate, error)  
    Delete(ctx context.Context, id uuid.UUID) error  
  
    List(ctx context.Context) ([]TaskTemplate, error)  
    ListActive(ctx context.Context) ([]TaskTemplate, error)  
}
```

Если генератор делает задачи окном, лучше добавить:

Go

Он достаёт только те шаблоны, которые потенциально могут породить задачи в этом окне:

```
is_active = true  
start_date <= to  
and (end_date is null or end_date >= from)
```

Репозиторий для Task

Обычный TaskRepository , но с методом для защиты от дублей:

```
type TaskRepository interface {
    Create(ctx context.Context, task Task) (Task, error)
    GetByID(ctx context.Context, id uuid.UUID) (Task, error)
    Update(ctx context.Context, task Task) (Task, error)
    Delete(ctx context.Context, id uuid.UUID) error

    ExistsByTemplateAndScheduledAt(
        ctx context.Context,
        templateID uuid.UUID,
        scheduledAt time.Time,
    ) (bool, error)

    CreateFromTemplateIfNotExists(
        ctx context.Context,
        template TaskTemplate,
        scheduledAt time.Time,
    ) error
}
```

В БД обязательно:

SQL

Иначе генератор рано или поздно наделает клонов. Люди изобрели unique constraints именно потому, что "ну мы же проверили в коде" не работает.

Что должно быть в сервисах

Я бы разделил на 3 части.

1. TaskService

Это сервис обычных задач:

```
type TaskService interface {
    CreateTask(ctx context.Context, input CreateTaskInput) (Task, error)
    GetTask(ctx context.Context, id uuid.UUID) (Task, error)
    UpdateTask(ctx context.Context, id uuid.UUID, input UpdateTaskInput) (Task, error)
```

```

DeleteTask(ctx context.Context, id uuid.UUID) error
}

```

Он занимается **конкретными задачами**:

```

создать разовую задачу
поменять статус
изменить описание
удалить задачу

```

То есть TaskService не должен знать много про периодичность.

2. TaskTemplateService

Вот тут создание "расписания".

```

type TaskTemplateService interface {
    CreateTemplate(ctx context.Context, input CreateTaskTemplateInput) (TaskTemplate,
    GetTemplate(ctx context.Context, id uuid.UUID) (TaskTemplate, error)
    UpdateTemplate(ctx context.Context, id uuid.UUID, input UpdateTaskTemplateInput)
    DeleteTemplate(ctx context.Context, id uuid.UUID) error
}

```

Он делает:

1. валидирует recurrence_type
2. парсит recurrence_config
3. проверяет interval_days > 0
4. проверяет times не пустой
5. проверяет формат времени "09:00"
6. проверяет day_of_month 1..30
7. проверяет start_date <= end_date
8. сохраняет шаблон

То есть пользователь создаёт не задачи, а правило:

```
"каждые 2 дня в 09:00 и 18:00"
```

А сервис сохраняет это как TaskTemplate .

3. RecurringTaskService / SchedulerService

Вот это уже генерация задач по расписанию:

```
type RecurringTaskService interface {
    Generate(ctx context.Context, from, to time.Time) error
}
```

Логика:

1. взять активные шаблоны
2. для каждого шаблона посчитать `scheduled_at` в окне `from..to`
3. создать `Task` для каждого `scheduled_at`
4. не создать дубль, если такая уже есть

Пример:

```
func (s *RecurringTaskService) Generate(ctx context.Context, from, to time.Time) error {
    templates, err := s.templateRepo.ListActiveForGeneration(ctx, from, to)
    if err != nil {
        return err
    }

    for _, template := range templates {
        slots, err := s.calculator.Calculate(template, from, to)
        if err != nil {
            return err
        }

        for _, scheduledAt := range slots {
            err := s.taskRepo.CreateFromTemplateIfNotExists(ctx, template, scheduledAt)
            if err != nil {
                return err
            }
        }
    }

    return nil
}
```

Отдельно: RecurrenceCalculator

Я бы вынес расчёт дат в отдельный компонент:

```
type RecurrenceCalculator interface {  
    Calculate(template TaskTemplate, from, to time.Time) ([]time.Time, error)  
  
    Go
```

Он не ходит в БД. Он просто считает:

шаблон + окно дат = список scheduled_at

Например:

```
DAILY, interval=2, times=[09:00, 18:00]  
start=2026-05-05  
window=2026-05-05..2026-05-10
```

результат:

```
2026-05-05 09:00  
2026-05-05 18:00  
2026-05-07 09:00  
2026-05-07 18:00  
2026-05-09 09:00  
2026-05-09 18:00
```

Итоговая структура

```
internal/task/  
    model.go  
    repository.go  
    service.go
```

```
internal/tasktemplate/  
    model.go  
    repository.go  
    service.go
```

```
internal/recurrence/  
    calculator.go  
    configs.go
```

```
internal/scheduler/  
    service.go
```

Или проще, если проект маленький:


```

internal/task/
  model.go
  template_model.go
  repository.go
  template_repository.go
  service.go
  template_service.go
  recurrence_calculator.go
  scheduler.go

```

Главное разделение

TaskRepository
просто работает с таблицей tasks

TaskTemplateRepository
просто работает с таблицей task_templates

TaskTemplateService
создаёт и валидирует расписание

RecurringTaskService
создаёт реальные задачи из расписания

RecurrenceCalculator
считает даты и времена

Самое важное: **создание расписания** = `TaskTemplateService.CreateTemplate`, а **создание задач по расписанию** = `RecurringTaskService.Generate`. Не смешивай это в один сервис, а то получится комбайн, который и валидирует JSON, и пишет в БД, и считает даты, и варит борщ.



```

type CreateInput struct {
    Title           string
    Description     string
    ReccurenceType  ReccurenceType
    ReccurenceConfog json.RawMessage
    StartDate      time.Time
    EndDate        *time.Time
}

```

Ну вот, почти хорошо... почти. Видно, что ты уже понимаешь, что нужно, но код чуть "пьяный после пары правок".

Давай аккуратно починим, пока это не превратилось в JSON-болото.

Проблемы в твоём коде

```
type CreateInput struct {  
    Title           string  
    Description     string  
    ReccurrenceType ReccurrenceType  
    ReccurrenceConfog json.RawMessage  
    StartDate      time.Time  
    EndDate        *time.Time  
}
```

1. Опечатки (и они реально бесят потом)

Reccurrence → Recurrence
Confog → Config

Go не простит тебе такие вещи, а ты потом будешь смотреть на код и думать "почему я страдаю".

2. Нет времён (times)

Сейчас у тебя есть только правило дат, но нет:

во сколько выполнять задачу

А мы уже выяснили, что:

2 раза в день — это нормально

3. Нет валидации структуры конфига

Ты используешь:

Go

— это ок, но значит, что дальше ты обязан это парсить руками в сервисе.

✅ Нормальная версия

```
type CreateInput struct {  
    Title      string  
    Description string  
  
    RecurrenceType RecurrenceType  
    RecurrenceConfig json.RawMessage  
  
    StartDate time.Time  
    EndDate   *time.Time  
}
```

🧠 Что происходит дальше (важно)

Этот `CreateInput` — это сырой ввод.

В сервисе ты должен сделать:

1. посмотреть `RecurrenceType`
2. распарсить `RecurrenceConfig` в нужную структуру
3. провалидировать

📦 Пример: DAILY

```
type DailyConfig struct {  
    IntervalDays int    `json:"interval_days"`  
    Times        []string `json:"times"`  
}
```

В сервисе

```
func (s *Service) CreateTemplate(ctx context.Context, input CreateInput) error {
    switch input.RecurrenceType {

    case Daily:
        var cfg DailyConfig

        if err := json.Unmarshal(input.RecurrenceConfig, &cfg); err != nil {
            return fmt.Errorf("invalid daily config: %w", err)
        }

        if cfg.IntervalDays <= 0 {
            return errors.New("interval_days must be > 0")
        }

        if len(cfg.Times) == 0 {
            return errors.New("at least one time required")
        }

        for _, t := range cfg.Times {
            if _, err := time.Parse("15:04", t); err != nil {
                return fmt.Errorf("invalid time format: %s", t)
            }
        }

        // всё ок → сохраняем

    default:
        return errors.New("unsupported recurrence type")
    }

    return nil
}
```

Маленький, но важный момент

Если хочешь сделать систему чуть менее “ломаемой”, можешь добавить helper:

```
Go
type TimeOfDay struct {
    Hour    int
    Minute  int
}
```

и хранить не строки "09:00" , а нормальную структуру.

Но для старта строки ок, не усложняй себе жизнь раньше времени.

⚠ Частая ошибка

Люди делают так:

```
Go
```

```
Times string // "09:00,18:00"
```

И потом парсят это руками как будто это CSV из 2005 года.

Не надо. У тебя уже есть JSON — используй его нормально:

```
JSON
```

```
"times": ["09:00", "18:00"]
```

💬 Итог

Твой `CreateInput` — нормальная заготовка, просто:

- исправь опечатки
- оставь `json.RawMessage`
- ВСЮ логику перенеси в сервис
- не забывай про `times`

И главное:

```
CreateInput ≠ готовая модель
```

Это просто сырой запрос, который сервис превращает в валидный `TaskTemplate` .

Если начнёшь туда пихать бизнес-логику — через пару дней сама себя возненавидишь.



```

package tasktemplate

import (
    "encoding/json"
    "time"

    "example.com/taskservice/internal/domain"
    "github.com/google/uuid"
)

type TaskTemplate struct {
    ID          uuid.UUID
    Title       string
    Description  string
    AssignedAt  uuid.UUID
    ReccurenceType    ReccurenceType
    ReccurenceConfog  json.RawMessage
    StartDate        time.Time
    EndDate          *time.Time
    Status           domain.Status
}

type ReccurenceType string

const (
    Daily          ReccurenceType = "DAILY"
    Monthly        ReccurenceType = "MONTHLY"
    SpecificDates  ReccurenceType = "SPECIFIC_DATES"
    DayParity      ReccurenceType = "DAY_PARITY"
)

сделай create и update inputs

```

Вот так нормальнее. И да, у тебя опечатки `Reccurence` / `Confog` лучше исправить сразу, пока они не расползлись по проекту, как плесень по влажному подвалу.

```
package tasktemplate
```

```

import (
    "encoding/json"
    "time"

    "example.com/taskservice/internal/domain"

    "github.com/google/uuid"
)

type CreateInput struct {
    Title      string      `json:"title"`
    Description string      `json:"description"`
    AssignedAt uuid.UUID    `json:"assigned_at"`

    RecurrenceType RecurrenceType `json:"recurrence_type"`
    RecurrenceConfig json.RawMessage `json:"recurrence_config"`

    StartDate time.Time `json:"start_date"`
    EndDate   *time.Time `json:"end_date,omitempty"`

    Status domain.Status `json:"status"`
}

type UpdateInput struct {
    Title      *string      `json:"title,omitempty"`
    Description *string      `json:"description,omitempty"`
    AssignedAt *uuid.UUID    `json:"assigned_at,omitempty"`

    RecurrenceType *RecurrenceType `json:"recurrence_type,omitempty"`
    RecurrenceConfig json.RawMessage `json:"recurrence_config,omitempty"`

    StartDate *time.Time `json:"start_date,omitempty"`
    EndDate   *time.Time `json:"end_date,omitempty"`

    Status *domain.Status `json:"status,omitempty"`
}

```

Но я бы назвала поле не `AssignedAt`, а `AssigneeID`.

`AssignedAt` звучит как "когда назначили", а не "кому назначили". Маленькая семантическая мина, зато потом красиво взрывается.

Лучше так:

Go

И в модели тоже:

Go

Ещё лучше поправить весь нейминг:

```
type RecurrenceType string

const (
    Daily      RecurrenceType = "DAILY"
    Monthly    RecurrenceType = "MONTHLY"
    SpecificDates RecurrenceType = "SPECIFIC_DATES"
    DayParity   RecurrenceType = "DAY_PARITY"
)
```

Status в TaskTemplate спорный: у шаблона обычно должен быть IsActive bool, а статус new/in_progress/done относится к конкретной задаче, не к расписанию. Тут лучше:

Go



не, а можешь json теги добавить в саму структуру, а create и update без тегов?

Конечно можешь. Раз уж ты решила быть последовательной, давай хотя бы без опечаток, а то Confog выглядит как крик о помощи.

Вот аккуратная версия.

Основная модель (с JSON тегами)

```
type TaskTemplate struct {
    ID          uuid.UUID    `json:"id"`
```



```

Title      string      `json:"title"`
Description string      `json:"description"`
AssigneeID uuid.UUID    `json:"assignee_id"`

RecurrenceType RecurrenceType `json:"recurrence_type"`
RecurrenceConfig json.RawMessage `json:"recurrence_config"`

StartDate time.Time `json:"start_date"`
EndDate   *time.Time `json:"end_date,omitempty"`

IsActive bool      `json:"is_active"`
Status   domain.Status `json:"status"`

CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
}

```



CreateInput (без тегов)

```

type CreateInput struct {
    Title      string
    Description string
    AssigneeID uuid.UUID

    RecurrenceType RecurrenceType
    RecurrenceConfig json.RawMessage

    StartDate time.Time
    EndDate   *time.Time

    Status domain.Status
}

```



UpdateInput (без тегов)

```

type UpdateInput struct {
    Title      *string
    Description *string
    AssigneeID *uuid.UUID

    RecurrenceType *RecurrenceType
    RecurrenceConfig *json.RawMessage
}

```

```
Go

EndDate    *time.Time

IsActive *bool
Status    *domain.Status
}
```

💡 Пара замечаний (чтобы ты потом не страдала)

- RecurrenceConfig в UpdateInput лучше делать *json.RawMessage , иначе ты не поймешь:

```
поле не передали
или передали пустое
```

- omitempty нужен только в API-моделях (DTO), а не обязательно в доменной модели. Но так жить проще.
- Status у шаблона — всё ещё сомнительная идея, но если у вас так в ТЗ, значит кто-то уже принял questionable решения за тебя.

🧠 Итог

```
модель → с json тегами (для API)
input'ы → без тегов (чисто внутренняя логика)
```

И да, теперь это выглядит как код, а не как случайный набор символов после дедлайна.

📄 👍 💬 ↗️ ↺ ...